

Construction Replay Transcript

EDI 835 → OCI Object Storage — Design · 2026-08-13 · what you'll hear in documents/construction-replay.mp4

What this interface does

Poll X12 835 remits from FTP, fork each ST / Transaction, map to JSON, and PUT-style-post each object to a local Object Storage mock (/n/{namespace}/b/{bucket}/o/{object}). Demo only — not Oracle IAM, not signed OCI requests.

Who it's for

- Sources: Trading-partner drop on FTP (sftp container, user demo / demo, dir upload)
- Destinations: JSON archive (output/json/), mock bucket (output/oci-received/ + HTTP), raw EDI archive
- Demo vs production: Demo only — stock HttpPostTransport against a local mock. Real OCI PutObject needs signed PUT (product gap).
- Anyone watching the construction video or reading this transcript

Systems involved

- Sources: Trading-partner drop on FTP (sftp container, user demo / demo, dir upload)
- Destinations: JSON archive (output/json/), mock bucket (output/oci-received/ + HTTP), raw EDI archive
- Demo vs production: Demo only — stock HttpPostTransport against a local mock. Real OCI PutObject needs signed PUT (product gap).

Documents we also produce

EDI835_OCI_Capability_Brief.pdf · ready

Short overview of what the interface does — shareable without reading route XML.

EDI835_OCI_Test_Plan.pdf · ready

Scenarios that prove the interface works end-to-end.

Module documentation pack (8 deep-dive PDFs) · ready

PilotFish product PDFs for the modules used in this build.

EDI835_OCI_V2_Route_Diagrams.pdf · ready

V2 route diagrams for visual review of module topology.

construction-replay.mp4 + this transcript · generate when ready

Narrated walkthrough of building the routes, then a live Demo-tab test when inject is available.

Narration

Spoken lines from the construction video (setup, build-replay, live test, closing).

Setup

Welcome

Welcome to the PilotFish demo: EDI 835 to OCI Object Storage. Poll X12 835 remits from FTP, fork each ST / Transaction, map to JSON, and PUT-style-post each object to a local Object Storage mock (/n/{namespace}/b/{bucket}/o/{object}). PilotFish supports the whole flow out of the box.

Pipeline overview

Before we wire anything, here's the flow end to end. Poll X12 835 remits from FTP, fork each ST / Transaction, map to JSON, and PUT-style-post each object to a local Object Storage mock (`/n/{namespace}/b/{bucket}/o/{object}`).

External systems

And here's what sits around the routes. FTP drop, Oci Mock, PilotFish eiPlatform, and Demo Web UI. All of those are spun up in docker images for this demo.

1 - SFTP Poll And Stage

First route

Poll X12 835 remits from FTP, fork each ST / Transaction, map to JSON, and PUT-style-post each object to a local Object Storage mock (`/n/{namespace}/b/{bucket}/o/{object}`). We'll build that in 2 routes. Starting with a blank canvas on the first route.

FTP listener

First up is the FTP listener — it watches the upload folder on the FTP server. We're only taking `.edi,835.txt` files. It checks about every 8 seconds. Once we've got the file, we delete it remotely so it doesn't get picked up again.

Archive step

Then we write the X12 text to disk. That lands under `/opt/pilotfish/output/archive`.

Stage-to-disk step

Then we write the file out to `/opt/pilotfish/output/staged`. That keeps FTP pickup separate from the next route.

2 - Split And Put Objects

Next route

Next route — blank canvas again, then the modules one at a time.

Staged-folder listener

Next is the staged-folder listener — it never talks to FTP, just the local stage. That folder is `/opt/pilotfish/output/staged`. We're only taking `.edi,835.txt` files. It checks about every 5 seconds.

Fork - XPath

This XPath fork splits the file so each transaction set becomes its own message.

Extract ST Control Number

XPath evaluation copies a few fields off the XML onto the transaction so later steps can use them.

Map Transaction To JSON

Now for the mapping — we're using the stock XSLT processor with `transaction-to-json.xslt`. It maps `st`, `clp`, `charge`.

Archive step

Then we write the JSON summary to disk. That lands under `/opt/pilotfish/output/json`.

Post JSON To Object Storage

Then we HTTP POST each JSON object to the Object Storage mock. After the response comes back, post-processors keep going on that same transaction.

Live test

After construction, the video switches to the Demo tab and runs a live inject when the UI supports it.

Prove it works

Routes are built. Let's switch to the Demo tab and prove it works.

Drop a sample

I'll drop this sample into the FTP upload folder — same place a trading partner would.

Waiting for the routes

Give the routes a moment to pick up the file and write the output.

JSON objects

There it is — output on the results panel.

Drop another sample

I'll drop another copy so we can watch it land again.

Waiting for the routes

Same path — the second result should show up here.

JSON objects

And there's the second result.

Closing

Demo complete

That's the demo — poll X12 835 remits from FTP, fork each ST / Transaction, map to JSON, and PUT-style-post each object to a local Object Storage mock. Thanks for choosing PilotFish.